

ECommerce.Services.G18 - Casos de Prueba

Este documento reúne casos de prueba manuales basados en los endpoints y reglas de negocio implementados en el repositorio.

Supuestos generales

- Solución compilada con **.NET 8**.
- Microservicios levantados localmente.
- Puertos esperados:
 - **Products.API**: <http://localhost:5001>
 - **Users.API**: <http://localhost:5002>
 - **Orders.API**: <http://localhost:5003>
 - **Cart.API**: <http://localhost:5004>
 - **Notifications.API**: <http://localhost:5005>
- Las bases SQLite se crean automáticamente al iniciar cada servicio.

Usuarios

TC-USR-001 - Registrar usuario correctamente

- **Identificador**: TC-USR-001
- **Requisito/Escenario asociado**: Alta de usuario válida en **POST** </api/users/register>
- **Precondiciones**: **Users.API** en ejecución.
- **Datos de entrada**:

```
{  
  
  "nombre": "Ana",  
  
  "apellido": "Lopez",  
  
  "email": "ana.lopez@test.com",  
  
  "password": "Clave123"  
}
```

- **Pasos**:
 1. Enviar **POST** a <http://localhost:5002/api/users/register>.
 2. Verificar el código HTTP.
 3. Guardar el **id** retornado para pruebas posteriores.

- **Resultado esperado:** Respuesta **201 Created** con los datos del usuario creado y sin exponer la contraseña.
- **Resultado obtenido:**

The screenshot shows a REST client interface with the following details:

- Request body:** A JSON object with fields: "nombre": "Ana", "apellido": "Lopez", "email": "ana.lopez@test.com", and "password": "Clave123".
- Execute:** A blue button to execute the request.
- Responses:** A section showing the response details.
 - Code:** 201 (undocumented).
 - Response body:** A JSON object with fields: "id": "19deb268-5288-4dec-94a5-5b21f7ca8e3e", "nombre": "Ana", "apellido": "Lopez", "email": "ana.lopez@test.com", "fechaRegistro": "2026-06-10T22:59:57.564593Z", and "activo": true.
 - Response headers:** content-type: application/json; charset=utf-8, date: Wed, 10 Jun 2026 22:59:57 GMT, location: /api/users/19deb268-5288-4dec-94a5-5b21f7ca8e3e, server: Kentrol, transfer-encoding: chunked, x-correlation-id: 491048ab-c5fb-4696-8908-ac6b54c31869.

TC-USR-002 - Rechazar registro inválido

- **Identificador:** TC-USR-002
- **Requisito/Escenario asociado:** Validación de registro de usuario
- **Precondiciones:** **Users.API** en ejecución.
- **Datos de entrada:**

```
{
  "nombre": "",
  "apellido": "Perez",
  "email": "correo-invalido",
  "password": "123"
```

}

- **Pasos:**

1. Enviar **POST** a <http://localhost:5002/api/users/register> con el payload inválido.
2. Revisar el cuerpo de error.

- **Resultado esperado:** Respuesta **400** con **errorCode** **USR-002** y mensaje indicando campos inválidos.

- **Resultado obtenido:**

The screenshot shows a REST client interface. At the top, a curl command is displayed: `curl -X 'POST' \n 'http://localhost:5002/api/users/register' \n -H 'accept: */*' \n -H 'Content-Type: application/json' \n -d '{\n "nombre": "",\n "apellido": "Perez",\n "email": "correo-invalido",\n "password": "123"\n}'`. Below this, the Request URL is `http://localhost:5002/api/users/register`. The Server response section shows a 400 status code with the message "Error: Bad Request". The Response body is a JSON object: `{\n "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",\n "title": "Error de Validaci\u00f3n",\n "status": 400,\n "detail": "El nombre es requerido.; El email no tiene un formato v\u00e1lido.; La contrase\u00f1a debe tener al menos 6 caracteres.",\n "instance": "/api/users/register",\n "errorCode": "USR-002",\n "errorMessage": "El nombre es requerido.; El email no tiene un formato v\u00e1lido.; La contrase\u00f1a debe tener al menos 6 caracteres.",\n "correlationId": "a69ad9dc-2f42-48be-8ca6-611275a37d15"\n}`. The Response headers section shows: `cache-control: no-cache,no-store\ncontent-type: application/json; charset=utf-8\ndate: Wed, 10 Jun 2026 23:16:59 GMT\nexpires: -1\npragma: no-cache\nserver: Mestral\ntransfer-encoding: chunked`. At the bottom, there is a table with columns "Code" and "Description", and a "Links" column.

TC-USR-003 - Rechazar email duplicado

- **Identificador:** TC-USR-003
- **Requisito/Escenario asociado:** Regla de unicidad de email
- **Precondiciones:** Existencia previa de un usuario con email ana.lopez@test.com.

- **Datos de entrada:** Mismo payload usado en **TC-USR-001**.

- **Pasos:**

1. Reenviar **POST** a <http://localhost:5002/api/users/register> con el mismo email.
2. Revisar el c\u00f3digo y cuerpo de respuesta.

- **Resultado esperado:** Respuesta **409** con **errorCode** **USR-001**.

- **Resultado obtenido:**

Parameters

Cancel

Reset

No parameters

Request body required

application/json

```
{
  "nombre": "Pablo",
  "apellido": "Crackencio",
  "email": "ana.lopez@test.com",
  "password": "Clave456"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5002/api/users/register' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre": "Pablo",
    "apellido": "Crackencio",
    "email": "ana.lopez@test.com",
    "password": "Clave456"
  }'
```

Request URL

http://localhost:5002/api/users/register

Server response

Code

Details

409

undocumented

Error: Conflict

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.9",
  "title": "Conflicto de Negocio",
  "status": 409,
  "detail": "El email 'ana.lopez@test.com' ya est\u00e1 registrado.",
  "instance": "/api/users/register",
  "errorCode": "USR-001",
  "errorMessage": "El email 'ana.lopez@test.com' ya est\u00e1 registrado.",
  "correlationId": "58c9c837-27a1-4f33-ae4f-8bdcb1b284b7"
}
```

Download

Response headers

```
cache-control: no-cache,no-store
content-type: application/json; charset=utf-8
date: Wed, 18 Jun 2020 23:23:18 GMT
expires: -1
pragma: no-cache
server: Kestrel
transfer-encoding: chunked
```

Responses

Code

Description

Links

TC-USR-004 - Bloquear cuenta tras tres intentos fallidos

- **Identificador:** TC-USR-004
- **Requisito/Escenario asociado:** Bloqueo por intentos fallidos en **POST** `/api/users/login`
- **Precondiciones:** Usuario activo existente con email v\u00e1lido.
- **Datos de entrada:**

```
{
  "email": "ana.lopez@test.com",
  "password": "PasswordIncorrecta"
}
```

- **Pasos:**

1. Enviar tres veces consecutivas **POST** a <http://localhost:5002/api/users/login> con contraseña inválida.
 2. Observar cada respuesta.
- **Resultado esperado:** Primer y segundo intento retornan **401** con **USR-003**; el tercer intento retorna **403** con **USR-004**.
 - **Resultado obtenido:**

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5002/api/users/login' \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "email": "ana.lopez@test.com",
    "password": "cualquiercosa"
  }'
```

Request URL

<http://localhost:5002/api/users/login>

Server response

Code	Details
403 Undocumented	Error: Forbidden Response body <pre>{ "type": "https://tools.ietf.org/html/rfc7231#section-6.5.3", "title": "Prohibido", "status": 403, "detail": "Su cuenta fue bloqueada por superar el máximo de intentos fallidos. Contacte a soporte.", "instance": "/api/users/login", "errorCode": "USR-004", "errorMessage": "Su cuenta fue bloqueada por superar el máximo de intentos fallidos. Contacte a soporte.", "correlationId": "ff76488d-4cb1-40e0-8912-8eb91e98c33b" }</pre> Response headers <pre>cache-control: no-cache,no-store content-type: application/json; charset=utf-8 date: Wed, 10 Jun 2026 23:25:25 GMT expires: -1 pragma: no-cache server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

TC-USR-005 - Login exitoso

- **Identificador:** TC-USR-005
- **Requisito/Escenario asociado:** Autenticación válida de usuario
- **Precondiciones:** Usuario activo existente con contraseña correcta.
- **Datos de entrada:**

```
{
  "email": "ana.lopez@test.com",
  "password": "Clave123"
}
```

- **Pasos:**
 1. Enviar **POST** a <http://localhost:5002/api/users/login>.
 2. Validar la respuesta.
- **Resultado esperado:** Respuesta **200 OK** con **id**, **nombre**, **apellido** y **email** del usuario.

- **Resultado obtenido:**

The screenshot displays a REST client interface with the following details:

- Execute** and **Clear** buttons at the top.
- Responses** section:

 - Curl:**

```
curl -X 'POST' \
'http://localhost:5002/api/users/login' \
-H 'accept: */*' \
-H 'content-type: application/json' \
-d '{
  "email": "ana_lopez@test.com",
  "password": "Clave123"
}'
```
 - Request URL:** `http://localhost:5002/api/users/login`
 - Server response:**
 - Code:** 200
 - Details:**
 - Response body:**

```
{
  "id": "0616ff38-f2cb-4c79-b24d-d056acd42ab7",
  "nombre": "Ana",
  "apellido": "Lopez",
  "email": "ana_lopez@test.com"
}
```
 - Response headers:**

```
content-type: application/json; charset=utf-8
date: Wed, 10 Jun 2026 23:25:53 GMT
server: Kestrel
transfer-encoding: chunked
x-correlation-id: 0a23213b-901f-442d-925c-e8616feb364e
```

- Responses table:**

Code	Description	Links
200	OK	No links

Productos

TC-PRD-001 - Crear producto correctamente

- **Identificador:** TC-PRD-001
- **Requisito/Escenario asociado:** Alta válida de producto en `POST /api/products`
- **Precondiciones:** `Products.API` en ejecución.
- **Datos de entrada:**

```
{
  "nombre": "Mouse Gamer",
  "descripcion": "Mouse RGB",
  "precio": 15000,
  "stock": 10,
  "categoria": "Perifericos"
}
```

- **Pasos:**

1. Enviar `POST` a `http://localhost:5001/api/products`.

2. Guardar el **id** generado.

- **Resultado esperado:** Respuesta **201 Created** con el producto persistido.
- **Resultado obtenido:**

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5001/api/products' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre": "Mouse Gamer",
    "descripcion": "Mouse RGB",
    "precio": 15000,
    "stock": 10,
    "categoria": "Perifericos"
  }'
```

Request URL

<http://localhost:5001/api/products>

Server response

Code	Details
201	undocumented Response body <pre>{ "id": "5034f5a1-bf3d-4744-9a29-3d34795d936f", "nombre": "Mouse Gamer", "descripcion": "Mouse RGB", "precio": 15000, "stock": 10, "categoria": "Perifericos", "fechaCreacion": "2026-06-10T23:27:25.4891504Z" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Wed, 10 Jun 2026 23:27:25 GMT location: /api/products/5034f5a1-bf3d-4744-9a29-3d34795d936f server: Kestrel transfer-encoding: chunked x-correlation-id: d1003237-4216-4e09-8b88-c7d0de509abb</pre>

Responses

Code	Description	Links
200	OK	No links

TC-PRD-002 - Rechazar producto inválido

- **Identificador:** TC-PRD-002
- **Requisito/Escenario asociado:** Validaciones de nombre, precio, stock y categoría
- **Precondiciones:** **Products.API** en ejecución.
- **Datos de entrada:**

```
{
  "nombre": "",
  "descripcion": "x",
  "precio": 0,
  "stock": -1,
  "categoria": ""
}
```

- **Pasos:**
 1. Enviar **POST** a **http://localhost:5001/api/products** con datos inválidos.
 2. Revisar la respuesta.
- **Resultado esperado:** Respuesta **400** con **errorCode** **PRD-002**.
- **Resultado obtenido:**

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5001/api/products' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre": "",
    "descripcion": "x",
    "precio": 0,
    "stock": 1,
    "categoria": ""
  }'
```

Request URL

http://localhost:5001/api/products

Server response

Code	Details
400	Error: Bad Request

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
  "title": "Error de Validación",
  "status": 400,
  "detail": "El nombre es requerido.; El precio debe ser mayor a 0.; El stock debe ser mayor o igual a 0.; La categoría es requerida.",
  "instance": "/api/products",
  "errorCode": "PRD-002",
  "errorMessage": "El nombre es requerido.; El precio debe ser mayor a 0.; El stock debe ser mayor o igual a 0.; La categoría es requerida.",
  "correlationId": "c95b7c80-68c8-45c6-9e33-dbc04ffc872a"
}
```

Response headers

```
cache-control: no-cache,no-store
content-type: application/json; charset=utf-8
date: Thu, 11 Jun 2026 10:57:54 GMT
expires: -1
pragma: no-cache
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
------	-------------	-------

TC-PRD-003 - Rechazar producto duplicado

- **Identificador:** TC-PRD-003
- **Requisito/Escenario asociado:** Regla de unicidad nombre + categoría
- **Precondiciones:** Existencia previa del producto **Mouse Gamer** en categoría **Perifericos**.
- **Datos de entrada:** Mismo payload de **TC-PRD-001**.
- **Pasos:**
 1. Repetir el **POST** de creación.
 2. Revisar código y mensaje.
- **Resultado esperado:** Respuesta **409** con **errorCode** **PRD-003**.
- **Resultado obtenido:**

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5001/api/products' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre": "Mouse Gamer",
    "descripcion": "Mouse RGB",
    "precio": 15000,
    "stock": 10,
    "categoria": "Perifericos"
  }'
```

Request URL

http://localhost:5001/api/products

Server response

Code	Details
409	Error: Conflict

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.9",
  "title": "Conflicto de Negocio",
  "status": 409,
  "detail": "Ya existe un producto con el nombre 'Mouse Gamer' en la categoria 'Perifericos'.",
  "instance": "/api/products",
  "errorCode": "990-003",
  "errorMessage": "Ya existe un producto con el nombre 'Mouse Gamer' en la categoria 'Perifericos'.",
  "correlationId": "9de203d4-6f0c-49f4-8db7-fa6c9726f122"
}
```

Response headers

```
cache-control: no-cache,no-store
content-type: application/json; charset=utf-8
date: Thu, 11 Jun 2026 23:28:41 GMT
expires: -1
pragma: no-cache
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
------	-------------	-------

TC-PRD-004 - Filtrar productos por categoría y nombre

- **Identificador:** TC-PRD-004
- **Requisito/Escenario asociado:** Consulta filtrada en `GET /api/products`
- **Precondiciones:** Existencia de productos en diferentes categorías y nombres.
- **Datos de entrada:** Query string `?categoria=Perifericos&nombre=Mouse`.
- **Pasos:**
 1. Enviar `GET` a `http://localhost:5001/api/products?categoria=Perifericos&nombre=Mouse`.
 2. Verificar el contenido de la lista.
- **Resultado esperado:** Respuesta `200` conteniendo solo productos que cumplan el filtro.
- **Resultado obtenido:**

GET /api/products

Parameters

Cancel

Name	Description
categoria string (query)	Perifericos
nombre string (query)	Mouse Gamer

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5001/api/products?categoria=Perifericos&nombre=Mouse20Gamer' \
  -H 'accept: */*'
```

Request URL

http://localhost:5001/api/products?categoria=Perifericos&nombre=Mouse20Gamer

Server response

Code	Details
200	Response body <pre>{ "id": "5034f5a1-bf3d-4744-9a29-3d34795d936f", "nombre": "Mouse Gamer", "descripcion": "Mouse RGB", "precio": 15000, "stock": 10, "categoria": "Perifericos", "fechaCreacion": "2026-06-10T20:27:25.4891504-03:00" }</pre> Download Response headers <pre>content-type: application/json; charset=utf-8 date: Thu, 11 Jun 2026 23:33:10 GMT server: Kestrel transfer-encoding: chunked x-correlation-id: 4529ee46-4971-42d3-84a5-2a2ac9668d93</pre>

Responses

Code	Description	Links
200	OK	No links

TC-PRD-005 - Impedir borrado de producto con órdenes activas

- **Identificador:** TC-PRD-005
- **Requisito/Escenario asociado:** Validación inter-servicio con **Orders.API**
- **Precondiciones:**
 - Producto existente.
 - Orden asociada al producto en estado **Pendiente** o **Confirmada**.
 - **Products.API** y **Orders.API** en ejecución.
- **Datos de entrada:** **id** de un producto con órdenes activas.
- **Pasos:**
 1. Enviar **DELETE** a **http://localhost:5001/api/products/{id}**.
 2. Revisar la respuesta.
- **Resultado esperado:** Respuesta **409** con **errorCode** **PRD-004**; el producto no debe eliminarse.
- **Resultado obtenido:**

Name	Description
id * required string(\$uuid) (path)	5034f5a1-bf3d-4744-9a29-3d34795d936f

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
'http://localhost:5001/api/products/5034f5a1-bf3d-4744-9a29-3d34795d936f' \
-H 'accept: */*'
```

Request URL

```
http://localhost:5001/api/products/5034f5a1-bf3d-4744-9a29-3d34795d936f
```

Server response

Code	Details
409 Undocumented	Error: Conflict Response body <pre>{ "type": "https://tools.ietf.org/html/rfc7231#section-6.5.9", "title": "Conflicto de Negocio", "status": 409, "detail": "El producto tiene \u00f3rdenes activas y no puede eliminarse.", "instance": "/api/products/5034f5a1-bf3d-4744-9a29-3d34795d936f", "errorCode": "PRD-00A", "errorMessage": "El producto tiene \u00f3rdenes activas y no puede eliminarse.", "correlationId": "0549ce11-8ef2-47d4-b074-dff05709617a" }</pre> Response headers <pre>cache-control: no-cache,no-store content-type: application/json; charset=utf-8 date: Thu, 11 Jun 2026 23:40:46 GMT expires: -1 pragma: no-cache server: Kestrel transfer-encoding: chunked</pre>

Download

Carrito

TC-CRT-001 - Crear carrito autom\u00e1ticamente al agregar primer \u00edtem

- **Identificador:** TC-CRT-001
- **Requisito/Escenario asociado:** Alta impl\u00edcita de carrito en **POST** `/api/cart/{userId}/items`
- **Precondiciones:**
 - **Cart.API** y **Products.API** en ejecuci\u00f3n.
 - Producto existente con stock suficiente.
 - **userId** sin carrito previo.
- **Datos de entrada:**

```
{

  "productId": "{productId}",

  "cantidad": 2

}
```

- **Pasos:**

1. Enviar **POST** a `http://localhost:5004/api/cart/{userId}/items`.
 2. Consultar luego **GET** `http://localhost:5004/api/cart/{userId}`.
- **Resultado esperado:** El **POST** responde exitosamente y el **GET** posterior retorna un carrito con el ítem agregado.
 - **Resultado obtenido:**

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5004/api/cart/900312cc-7118-45e9-9abd-7d99542f632/items' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "productId": "5034f5a1-bf3d-4744-9a29-3d34795d936f",
    "cantidad": 1
  }'
```

Request URL

`http://localhost:5004/api/cart/900312cc-7118-45e9-9abd-7d99542f632/items`

Server response

Code	Details
200	Response body <pre>{ "usuarioId": "900312cc-7118-45e9-9abd-7d99542f632", "items": [{ "productId": "5034f5a1-bf3d-4744-9a29-3d34795d936f", "cantidad": 1 }], "fechaActualizacion": "2026-06-11T20:48:56.6592538-03:00" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Thu, 11 Jun 2026 23:48:56 GMT server: Restful transfer-encoding: chunked x-correlation-id: b0e83896-111e-46b2-ab71-63ea2901fd61</pre>

Responses

TC-CRT-002 - Rechazar cantidad inválida en carrito

- **Identificador:** TC-CRT-002
- **Requisito/Escenario asociado:** Validación de cantidad en carrito
- **Precondiciones:** `Cart.API` en ejecución.
- **Datos de entrada:**

```
{

  "productId": "{productId}",

  "cantidad": 0

}
```

- **Pasos:**
 1. Enviar **POST** a `http://localhost:5004/api/cart/{userId}/items` con cantidad `0`.
 2. Revisar la respuesta.
- **Resultado esperado:** Respuesta `400` con `errorCode CRT-004`.
- **Resultado obtenido:**

Curl

```
curl -X 'POST' \
  'http://localhost:5004/api/cart/900312cc-7118-45e9-9abd-7dd99542f632/items' \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "productId": "5034f5a1-bf3d-4744-9a29-3d34795d936f",
    "cantidad": 0
  }'
```

Request URL

<http://localhost:5004/api/cart/900312cc-7118-45e9-9abd-7dd99542f632/items>

Server response

Code	Details
409	Error: Conflict

Undocumented

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.9",
  "title": "conflicto de Negocio",
  "status": 409,
  "detail": "Cantidad inválida.",
  "instance": "/api/cart/900312cc-7118-45e9-9abd-7dd99542f632/items",
  "errorCode": "CRT-004",
  "errorMessage": "Cantidad inválida.",
  "correlationId": "9a304a7e-b4c1-4410-9153-1ce0c4ace9aa"
}
```

Response headers

```
cache-control: no-cache,no-store
content-type: application/json; charset=utf-8
date: Thu, 11 Jun 2026 23:50:23 GMT
expires: -1
pragma: no-cache
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description
------	-------------

Links

TC-CRT-003 - Rechazar agregado por stock insuficiente acumulado

- **Identificador:** TC-CRT-003
- **Requisito/Escenario asociado:** Validación de stock considerando cantidad ya agregada
- **Precondiciones:**
 - Producto existente con stock 3.
 - Carrito del usuario ya contiene 2 unidades de ese producto.
- **Datos de entrada:**

```
{
  "productId": "{productId}",
  "cantidad": 2
}
```

- **Pasos:**
 1. Enviar **POST** a <http://localhost:5004/api/cart/{userId}/items>.
 2. Revisar la respuesta.
- **Resultado esperado:** Respuesta 422 con **errorCode CRT-003** porque el acumulado supera el stock.
- **Resultado obtenido:**

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5004/api/cart/900312cc-7118-45e9-9abd-7dd99542f632/items' \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "productId": "23698b19-0abb-432e-94b5-512fcbcb40b5",
    "cantidad": 2
  }'
```

Request URL

http://localhost:5004/api/cart/900312cc-7118-45e9-9abd-7dd99542f632/items

Server response

Code	Details
422	Error: Unprocessable Entity

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc4918#section-11.2",
  "title": "Entidad no procesable",
  "status": 422,
  "detail": "Stock insuficiente. Disponible: 3, solicitado: 4.",
  "instance": "/api/cart/900312cc-7118-45e9-9abd-7dd99542f632/items",
  "errorCode": "CRT-003",
  "errorMessage": "Stock insuficiente. Disponible: 3, solicitado: 4.",
  "correlationId": "7f0ec663-f74f-43c3-918f-9eb479357ed8"
}
```

Response headers

```
cache-control: no-cache,no-store
content-type: application/json; charset=utf-8
date: Thu, 11 Jun 2026 23:58:19 GMT
expires: -1
pragma: no-cache
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description	Links
------	-------------	-------

TC-CRT-004 - Eliminar ítem existente del carrito

- **Identificador:** TC-CRT-004
- **Requisito/Escenario asociado:** Baja de ítem en **DELETE** `/api/cart/{userId}/items/{productId}`
- **Precondiciones:** Carrito existente con el producto cargado.
- **Datos de entrada:** `userId` y `productId` válidos.
- **Pasos:**
 1. Enviar **DELETE** a `http://localhost:5004/api/cart/{userId}/items/{productId}`.
 2. Consultar luego el carrito.
- **Resultado esperado:** Respuesta exitosa y el producto ya no aparece en el carrito.
- **Resultado obtenido:**

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://localhost:5004/api/cart/0016ff38-f2cb-4c79-b24d-dd56acd42ab7/items/76e3a2f2-52fc-4275-b63f-721a3d9c4d00' \
  -H 'accept: */*'
```

Request URL

http://localhost:5004/api/cart/0016ff38-f2cb-4c79-b24d-dd56acd42ab7/items/76e3a2f2-52fc-4275-b63f-721a3d9c4d00

Server response

Code	Details
204	

Response headers

```
date: Fri, 12 Jun 2026 00:21:42 GMT
server: Kestrel
x-correlation-id: 4211f8c3-9b99-474b-bde1-4ebfe7d23a45
```

Responses

Code	Description	Links
200	OK	No links

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5004/api/cart/0016ff38-f2cb-4c79-b24d-dd56acd42ab7' \
  -H 'accept: */*'
```

Request URL

http://localhost:5004/api/cart/0016ff38-f2cb-4c79-b24d-dd56acd42ab7

Server response

Code	Details
200	Response body <pre>{ "userId": "0016ff38-f2cb-4c79-b24d-dd56acd42ab7", "items": [], "fechaActualizacion": "2026-06-11T21:21:42.8215414-03:00" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Fri, 12 Jun 2026 00:22:26 GMT server: Kestrel transfer-encoding: chunked x-correlation-id: 898606fc-8c23-473c-af86-0071405cd785</pre>

Responses

Code	Description	Links
200	OK	No links

Órdenes

TC-ORD-001 - Crear orden correctamente

- **Identificador:** TC-ORD-001
- **Requisito/Escenario asociado:** Orquestación exitosa en **POST /api/orders**
- **Precondiciones:**
 - Usuario existente.
 - Producto(s) existente(s) con stock suficiente.
 - **Users.API, Products.API, Orders.API y Notifications.API** en ejecución.
- **Datos de entrada:**

```
{

  "userId": "{userId}",

  "items": [

    {

      "productId": "{productId}",

      "cantidad": 1

    }

  ]

}
```

- **Pasos:**
 1. Enviar **POST** a <http://localhost:5003/api/orders>.
 2. Guardar el **id** de la orden.
 3. Consultar luego la orden creada.
- **Resultado esperado:** Respuesta **201 Created**; la orden queda en estado **Confirmada** y el total se calcula con el precio actual del producto.
- **Resultado obtenido:**

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5003/api/orders' \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "userId": "900312cc-7118-45e9-9abd-7dd99542f632",
    "items": [
      {
        "productId": "5034f5a1-bf3d-4744-9a29-3d34795d936f",
        "cantidad": 1
      }
    ]
  }'
```

Request URL

<http://localhost:5003/api/orders>

Server response

Code	Details
201	Response body <pre>{ "id": "729d9a45-5950-4549-9324-cdea5f7632eb", "userId": "900312cc-7118-45e9-9abd-7dd99542f632", "items": [{ "productId": "5034f5a1-bf3d-4744-9a29-3d34795d936f", "cantidad": 1, "precioUnitario": 15000 }], "total": 15000, "estado": "Confirmada", "fechaCreacion": "2026-06-11T23:38:39.359932Z" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Thu, 11 Jun 2026 23:38:39 GMT location: /api/orders/729d9a45-5950-4549-9324-cdea5f7632eb server: Westrel transfer-encoding: chunked x-correlation-id: a5fb2595-8c9e-431b-91d5-ec2832f2ebbd</pre>

Responses

TC-ORD-002 - Rechazar orden con usuario inexistente

- **Identificador:** TC-ORD-002
- **Requisito/Escenario asociado:** Validación remota de usuario
- **Precondiciones:** **Orders.API** y **Users.API** en ejecución.
- **Datos de entrada:** **userId** inexistente y producto válido.
- **Pasos:**
 1. Enviar **POST** a <http://localhost:5003/api/orders> con un **userId** inexistente.
 2. Revisar la respuesta.
- **Resultado esperado:** Respuesta **404** con **errorCode** **ORD-003**.
- **Resultado obtenido:**

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:5003/api/orders' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "userId": "11111111-1111-1111-1111-111111111111",
    "items": [
      {
        "productId": "76e3a2f2-52fc-4275-b63f-721a3d9c4d00",
        "cantidad": 1
      }
    ]
  }'
```

Request URL

http://localhost:5003/api/orders

Server response

Code	Details
404	Error: Not Found

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.4",
  "title": "Recurso no encontrado",
  "status": 404,
  "detail": "Usuario no encontrado al crear la orden.",
  "instance": "/api/orders",
  "errorCode": "ORD-003",
  "errorMessage": "Usuario no encontrado al crear la orden.",
  "correlationId": "bc878ecf-f222-45be-b557-264953b81887"
}
```

Response headers

```
cache-control: no-cache,no-store
content-type: application/json; charset=utf-8
date: Fri, 12 Jun 2026 00:00:04 GMT
expires: -1
pragma: no-cache
server: Kestrel
transfer-encoding: chunked
```

Responses

TC-ORD-003 - Rechazar orden por producto inexistente

- **Identificador:** TC-ORD-003
- **Requisito/Escenario asociado:** Validación remota de producto
- **Precondiciones:** `Orders.API` y `Products.API` en ejecución.
- **Datos de entrada:** `productId` inexistente y usuario válido.
- **Pasos:**
 1. Enviar `POST` a `http://localhost:5003/api/orders` con un producto inexistente.
 2. Revisar la respuesta.
- **Resultado esperado:** Respuesta `404` con `errorCode` `ORD-004`.
- **Resultado obtenido:**

Curl

```
curl -X 'POST' \
  'http://localhost:5003/api/orders' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "userId": "0016ff38-f2cb-4c79-b04d-dd56ac042ab7",
    "items": [
      {
        "productId": "22222222-2222-2222-2222-222222222222",
        "cantidad": 1
      }
    ]
  }'
```

Request URL

http://localhost:5003/api/orders

Server response

Code	Details
404	Error: Not Found

Response body

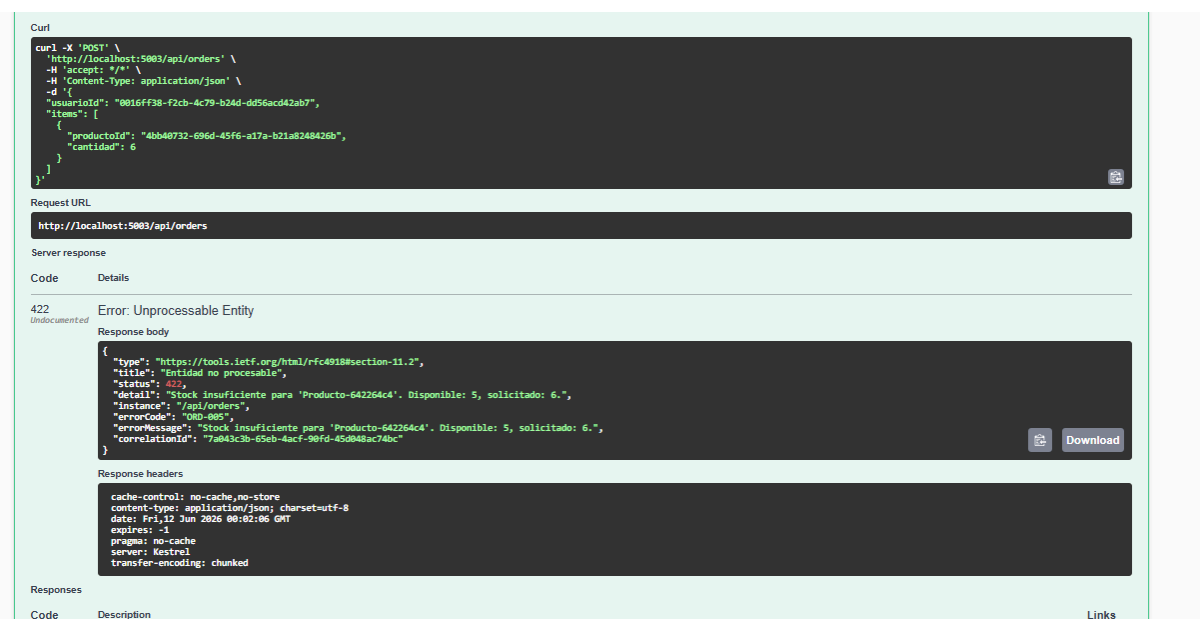
```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.4",
  "title": "Recurso no encontrado",
  "status": 404,
  "detail": "Producto no encontrado al crear la orden.",
  "instance": "/api/orders",
  "errorCode": "ORD-004",
  "errorMessage": "Producto no encontrado al crear la orden.",
  "correlationId": "50bb8dbb-0b84-4f3f-9b04-8ca30c822333"
}
```

Response headers

```
cache-control: no-cache,no-store
content-type: application/json; charset=utf-8
date: Fri, 12 Jun 2026 00:01:11 GMT
expires: -1
pragma: no-cache
server: Kestrel
transfer-encoding: chunked
```

TC-ORD-004 - Rechazar orden por stock insuficiente

- **Identificador:** TC-ORD-004
- **Requisito/Escenario asociado:** Validación de stock al crear orden
- **Precondiciones:** Producto existente con stock menor a la cantidad solicitada.
- **Datos de entrada:** Payload de orden con cantidad superior al stock disponible.
- **Pasos:**
 1. Enviar **POST** a <http://localhost:5003/api/orders>.
 2. Revisar el cuerpo de error.
- **Resultado esperado:** Respuesta **422** con **errorCode** **ORD-005**.
- **Resultado obtenido:**



Curl

```
curl -X 'POST' \
  'http://localhost:5003/api/orders' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "usuarioId": "0016ff38-f2cb-4c79-b24d-d456acd42ab7",
    "items": [
      {
        "productoId": "4bb40732-696d-45f6-a17a-b21a824842eb",
        "cantidad": 6
      }
    ]
  }'
```

Request URL

<http://localhost:5003/api/orders>

Server response

Code Details

422 Error: Unprocessable Entity

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc4918#section-11.2",
  "title": "Entidad no procesable",
  "status": 422,
  "detail": "Stock insuficiente para 'Producto-642264c4'. Disponible: 5, solicitado: 6.",
  "instance": "/api/orders",
  "errorCode": "ORD-005",
  "errorMessage": "Stock insuficiente para 'Producto-642264c4'. Disponible: 5, solicitado: 6.",
  "correlationId": "7a043c3b-65eb-4acf-90fd-45d048ac74bc"
}
```

Response headers

```
cache-control: no-cache, no-store
content-type: application/json; charset=utf-8
date: Fri, 12 Jun 2026 00:02:06 GMT
expires: -1
pragma: no-cache
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description
------	-------------

Links

TC-ORD-005 - Verificar descuento de stock luego de crear orden

- **Identificador:** TC-ORD-005
- **Requisito/Escenario asociado:** Consistencia entre **Orders.API** y **Products.API**
- **Precondiciones:** Producto con stock conocido, por ejemplo **10** unidades.
- **Datos de entrada:** Orden por **2** unidades de ese producto.
- **Pasos:**
 1. Consultar el producto antes de crear la orden.
 2. Crear la orden desde **Orders.API**.
 3. Volver a consultar el producto en **Products.API**.
- **Resultado esperado:** El stock final del producto disminuye exactamente en la cantidad ordenada.
- **Resultado obtenido:**

Curl

```
curl -X 'POST' \
  'http://localhost:5003/api/orders' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "usuarioId": "0016ff38-f2cb-4c79-b24d-dd56acd42ab7",
    "items": [
      {
        "productoId": "76e3a2f2-52fc-4275-b63f-721a3d9c4d00",
        "cantidad": 2
      }
    ]
  }'
```

Request URL

http://localhost:5003/api/orders

Server response

Code	Details
201	Response body <pre>{ "id": "b46ea73b-b573-4857-a8c2-feda95fe5218", "usuarioId": "0016ff38-f2cb-4c79-b24d-dd56acd42ab7", "items": [{ "productoId": "76e3a2f2-52fc-4275-b63f-721a3d9c4d00", "cantidad": 2, "precioUnitario": 15000 }], "total": 30000, "estado": "Confirmada", "fechaCreacion": "2026-06-12T00:03:48.1898728Z" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Fri, 12 Jun 2026 00:03:47 GMT location: /api/orders/b46ea73b-b573-4857-a8c2-feda95fe5218 server: Kestrel transfer-encoding: chunked x-correlation-id: e489cab2-ddfd-4a75-99cf-7315826d388c</pre>

Responses

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5001/api/products/76e3a2f2-52fc-4275-b63f-721a3d9c4d00' \
  -H 'accept: */*' \
```

Request URL

http://localhost:5001/api/products/76e3a2f2-52fc-4275-b63f-721a3d9c4d00

Server response

Code	Details
200	Response body <pre>{ "id": "76e3a2f2-52fc-4275-b63f-721a3d9c4d00", "nombre": "Mouse Gamer", "descripcion": "Mouse RGB", "precio": 15000, "stock": 0, "categoria": "Perifericos", "fechaCreacion": "2026-06-10T20:27:31.6864128-03:00" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Fri, 12 Jun 2026 00:05:08 GMT server: Kestrel transfer-encoding: chunked x-correlation-id: 3bdefb77-3f89-4b0a-a776-5d681a9178ad</pre>

Responses

Code	Description	Links
200	OK	No links

TC-ORD-006 - Validar transición de estado permitida

- **Identificador:** TC-ORD-006
- **Requisito/Escenario asociado:** Cambio válido de estado en **PUT** `/api/orders/{id}/status`
- **Precondiciones:** Orden existente en estado **Confirmada**.
- **Datos de entrada:**

```
{
```

```
  "nuevoEstado": "Enviada"
```

```
}
```

- **Pasos:**
 1. Enviar PUT a `http://localhost:5003/api/orders/{id}/status`.
 2. Consultar la orden actualizada.
- **Resultado esperado:** Respuesta 200 OK y orden con estado Enviada.
- **Resultado obtenido:**

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:5003/api/orders/b46ea73b-b573-4857-a0c2-feda95fe5218/status' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nuevoEstado": "Enviada"
  }'
```

Request URL

`http://localhost:5003/api/orders/b46ea73b-b573-4857-a0c2-feda95fe5218/status`

Server response

Code	Details
200	Response body <pre>{ "id": "b46ea73b-b573-4857-a0c2-feda95fe5218", "estado": "Enviada", "fechaActualizacion": "2026-06-12T00:11:28.9769777Z" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Fri, 12 Jun 2026 00:11:28 GMT server: Kestrel transfer-encoding: chunked x-correlation-id: c0c944d8-e237-45c0-86af-acb61b76bfc8</pre>

Responses

Code	Description	Links
200	OK	No links

TC-ORD-007 - Rechazar transición de estado inválida

- **Identificador:** TC-ORD-007
- **Requisito/Escenario asociado:** Regla de negocio de transición de estados
- **Precondiciones:** Orden existente en estado Pendiente.
- **Datos de entrada:**

```
{
  "nuevoEstado": "Entregada"
}
```

- **Pasos:**
 1. Enviar PUT a `http://localhost:5003/api/orders/{id}/status`.
 2. Revisar la respuesta.
- **Resultado esperado:** Respuesta 409 con `errorCode` ORD-006.
- **Resultado obtenido:**

The screenshot displays a REST client interface with the following details:

- Curl:**

```
curl -X 'PUT' \
  'http://localhost:5003/api/orders/9014b463-37f9-414b-8528-10f097efdacd/status' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nuevoEstado": "Entregada"
  }'
```
- Request URL:** `http://localhost:5003/api/orders/9014b463-37f9-414b-8528-10f097efdacd/status`
- Server response:**
 - Code:** 409
 - Details:** Error: Conflict
- Response body:**

```
{
  "type": "https://tools.ietf.org/html/rfc7231section-6.5.9",
  "title": "Conflicto de Negocio",
  "status": 409,
  "detail": "Una orden en estado 'Pendiente' no puede pasar a 'Entregada'.",
  "instance": "/api/orders/9014b463-37f9-414b-8528-10f097efdacd/status",
  "errorCode": "000-000",
  "errorMessage": "Una orden en estado 'Pendiente' no puede pasar a 'Entregada'.",
  "correlationId": "Ba734f83-39c6-4675-b70d-f496b20b03ea"
}
```
- Response headers:**

```
cache-control: no-cache, no-store
content-type: application/json; charset=utf-8
date: Fri, 12 Jun 2026 00:12:28 GMT
expires: -1
pragma: no-cache
server: Kestrel
transfer-encoding: chunked
```

TC-ORD-008 - La falla de notificación no debe anular la orden

- **Identificador:** TC-ORD-008
- **Requisito/Escenario asociado:** Tolerancia a fallo de `Notifications.API`
- **Precondiciones:**
 - `Users.API`, `Products.API` y `Orders.API` en ejecución.
 - `Notifications.API` detenido o inaccesible.
 - Usuario y producto válidos.
- **Datos de entrada:** Payload válido de creación de orden.
- **Pasos:**
 1. Crear una orden desde `Orders.API` con `Notifications.API` fuera de servicio.
 2. Consultar la orden creada.
- **Resultado esperado:** La creación de la orden sigue siendo exitosa; la orden queda confirmada aunque la notificación falle.
- **Resultado obtenido:**

Curl

```
curl -X 'POST' \
  'http://localhost:5003/api/orders' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "userId": "0016ff38-f2cb-4c79-b24d-dd56acd42ab7",
    "items": [
      {
        "productId": "76e3a2f2-52fc-4275-b63f-721a3d9c4d00",
        "cantidad": 1
      }
    ]
  }'
```

Request URL

http://localhost:5003/api/orders

Server response

Code Details

201
Undocumented

Response body

```
{
  "id": "97bde66-6b1d-4287-b135-8a304bc9c144",
  "userId": "0016ff38-f2cb-4c79-b24d-dd56acd42ab7",
  "items": [
    {
      "productId": "76e3a2f2-52fc-4275-b63f-721a3d9c4d00",
      "cantidad": 1,
      "precioUnitario": 15000
    }
  ],
  "total": 15000,
  "estado": "Confirmada",
  "fechaCreacion": "2026-06-12T00:15:22.6164157Z"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Fri, 12 Jun 2026 00:15:22 GMT
location: /api/orders/97bde66-6b1d-4287-b135-8a304bc9c144
server: Kestrel
transfer-encoding: chunked
x-correlation-id: ea3c11ba-b1b9-4e7d-83b1-eb42925ae586
```

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5003/api/orders/97bde66-6b1d-4287-b135-8a304bc9c144' \
  -H 'accept: */*' \
```

Request URL

http://localhost:5003/api/orders/97bde66-6b1d-4287-b135-8a304bc9c144

Server response

Code Details

200

Response body

```
{
  "id": "97bde66-6b1d-4287-b135-8a304bc9c144",
  "userId": "0016ff38-f2cb-4c79-b24d-dd56acd42ab7",
  "items": [
    {
      "productId": "76e3a2f2-52fc-4275-b63f-721a3d9c4d00",
      "cantidad": 1,
      "precioUnitario": 15000
    }
  ],
  "total": 15000,
  "estado": "Confirmada",
  "fechaCreacion": "2026-06-11T21:15:22.6164157-03:00"
}
```

Response headers

```
content-type: application/json; charset=utf-8
date: Fri, 12 Jun 2026 00:16:39 GMT
server: Kestrel
transfer-encoding: chunked
x-correlation-id: da1088e2-3b12-49ef-ac43-f843f9ca506e
```

Responses

Notificaciones

TC-NTF-001 - Enviar notificación correctamente

- **Identificador:** TC-NTF-001
- **Requisito/Escenario asociado:** Alta de notificación válida en **POST /api/notifications/send**
- **Precondiciones:**
 - Usuario existente.
 - **Notifications.API** y **Users.API** en ejecución.
- **Datos de entrada:**

```
{

    "usuarioid": "{userId}",

    "mensaje": "Su orden fue confirmada.",

    "tipo": "Email"

}
```

- **Pasos:**

1. Enviar **POST** a <http://localhost:5005/api/notifications/send>.
2. Consultar luego **GET** [/api/notifications/{userId}](http://localhost:5005/api/notifications/{userId}).

- **Resultado esperado:** Respuesta exitosa y la notificación aparece listada para el usuario.

- **Resultado obtenido:**

The screenshot shows a web browser interface for testing API requests. The 'Curl' section displays the command used to send a POST request. The 'Request URL' is <http://localhost:5005/api/notifications/send>. The 'Server response' section shows a 201 status code and a JSON response body. The response body contains the following data:

```
{
  "id": "4f2c7847-4d2b-4d69-a88d-d73df67ae176",
  "usuarioid": "0016ff38-f2cb-4c79-b24d-d56acd42ab7",
  "mensaje": "Su orden fue confirmada.",
  "tipo": "Email",
  "estado": "Enviada",
  "fechaEnvio": "2026-06-11T23:45:04.4874861Z"
}
```

The 'Response headers' section shows the following headers:

```
content-type: application/json; charset=utf-8
date: Thu, 11 Jun 2026 23:45:03 GMT
location: /api/notifications/4f2c7847-4d2b-4d69-a88d-d73df67ae176
server: Kestrel
transfer-encoding: chunked
x-correlation-id: 008ccc51-36a1-42ab-9ea8-a1a8761849d8
```

The 'Responses' table at the bottom shows a single response with a 200 status code and the description 'OK'.

Code	Description	Links
200	OK	No links

TC-NTF-002 - Rechazar notificación con tipo inválido

- **Identificador:** TC-NTF-002
- **Requisito/Escenario asociado:** Validación de tipo y mensaje en notificaciones
- **Precondiciones:** [Notifications.API](#) en ejecución.
- **Datos de entrada:**

```
{

    "usuarioid": "{userId}",

    "mensaje": "Mensaje de prueba",

    "tipo": "Email"

}
```

"tipo": "FAX"

}

- **Pasos:**

1. Enviar **POST** a <http://localhost:5005/api/notifications/send>.
2. Revisar la respuesta.

- **Resultado esperado:** Respuesta **400** con **errorCode** NTF-002.

- **Resultado obtenido:**

The screenshot shows a web browser interface with the following sections:

- Responses**: A table with columns 'Code' and 'Description'. It shows a 400 status code with the description 'Error: Bad Request'.
- Request URL**: A text box containing 'http://localhost:5005/api/notifications/send'.
- Server response**: A section showing the response body as a JSON object:

```
{  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",  "title": "Error de Validaci\u00f3n",  "status": 400,  "detail": "Tipo de notificaci\u00f3n no reconocido. Debe ser Email, Push o SMS.",  "instance": "/api/notifications/send",  "errorCode": "NTF-002",  "errorMessage": "Tipo de notificaci\u00f3n no reconocido. Debe ser Email, Push o SMS.",  "correlationId": "a21b493b-cea1-4626-a959-54ec5a3f4e8f"}
```
- Response headers**: A section showing the response headers as a text block:

```
cache-control: no-cache, no-store
content-type: application/json; charset=utf-8
date: Thu, 11 Jun 2026 23:47:55 GMT
expires: -1
pragma: no-cache
server: Kestrel
transfer-encoding: chunked
```

Observabilidad e integraci\u00f3n

TC-OBS-001 - Generar y devolver Correlation ID

- **Identificador:** TC-OBS-001
- **Requisito/Escenario asociado:** Trazabilidad mediante **X-Correlation-Id**
- **Precondiciones:** Cualquier API en ejecuci\u00f3n.
- **Datos de entrada:** Request sin header **X-Correlation-Id**.
- **Pasos:**
 1. Enviar una solicitud **GET** a cualquier endpoint, por ejemplo <http://localhost:5001/api/products>.
 2. Revisar los headers de la respuesta.
- **Resultado esperado:** La respuesta devuelve un header **X-Correlation-Id** generado autom\u00e1ticamente.
- **Resultado obtenido:**

Curl

```
curl -X 'POST' \
  'http://localhost:5001/api/products' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "nombre": "string",
    "descripcion": "string",
    "precio": 0,
    "stock": 0,
    "categoria": "string"
  }'
```

Request URL

http://localhost:5001/api/products

Server response

Code	Details
400	Error: Bad Request

Undocumented

Response body

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.1",
  "title": "Error de Validación",
  "status": 400,
  "detail": "El precio debe ser mayor a 0.",
  "instance": "/api/products",
  "errorCode": "PBD-002",
  "errorMessage": "El precio debe ser mayor a 0.",
  "correlationId": "3692e61e-57ff-4390-9fee-e490ffdc35c"
}
```

Response headers

```
cache-control: no-cache,no-store
content-type: application/json; charset=utf-8
date: Thu, 11 Jun 2026 23:49:52 GMT
expires: -1
pragma: no-cache
server: Kestrel
transfer-encoding: chunked
```

Responses

Code	Description
------	-------------

Links

TC-OBS-002 - Propagar Correlation ID enviado por el cliente

- **Identificador:** TC-OBS-002
- **Requisito/Escenario asociado:** Preservación de trazabilidad en llamadas inter-servicio
- **Precondiciones:** `Orders.API`, `Users.API` y `Products.API` en ejecución.
- **Datos de entrada:** Request a creación de orden con header `X-Correlation-Id: QA-TRACE-001`.
- **Pasos:**
 1. Enviar `POST` a `http://localhost:5003/api/orders` con el header indicado.
 2. Revisar la respuesta y logs relacionados.
- **Resultado esperado:** La respuesta conserva el mismo `X-Correlation-Id` y los logs asociados usan ese identificador.
- **Resultado obtenido:**

POST http://localhost:5003/

http://localhost:5003/api/orders

POST http://localhost:5003/api/orders

Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

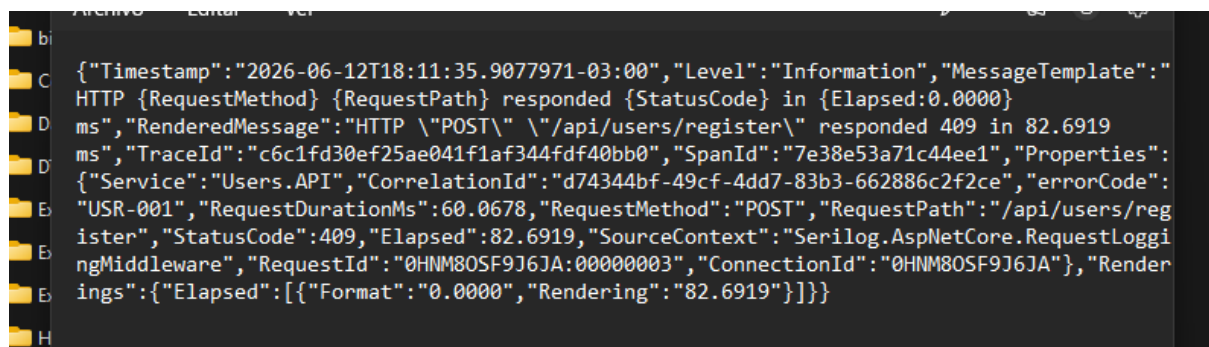
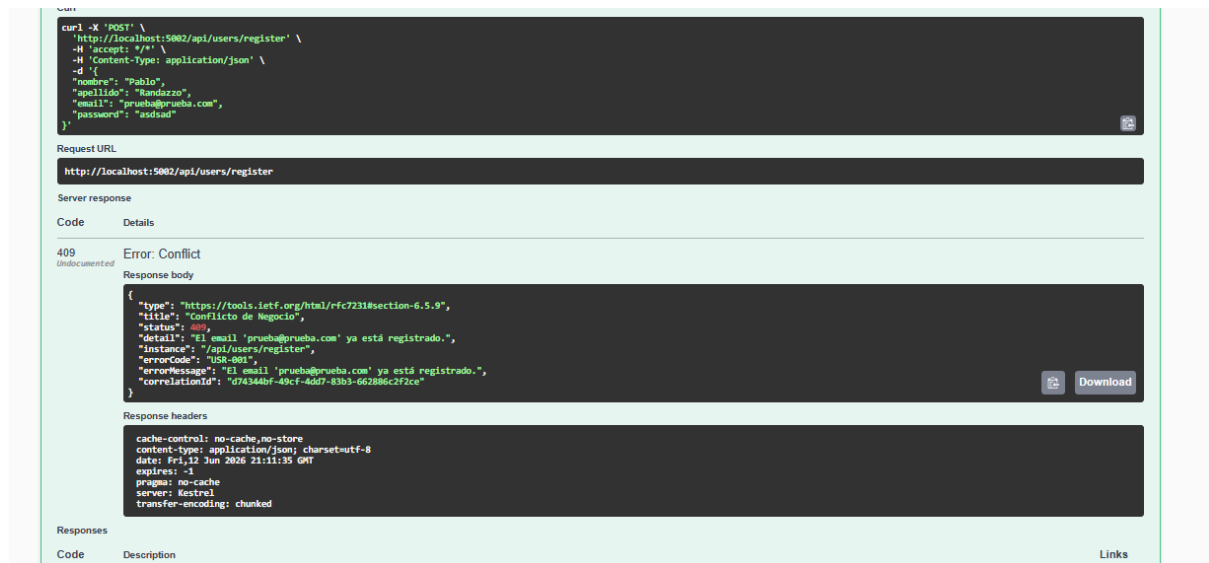
```
1 {
2   "usuarioId": "900312cc-7118-45e9-9abd-7dd99542f632",
3   "items": [
4     {
5       "productoId": "5034f5a1-bf3d-4744-9a29-3d34795d936f",
6       "cantidad": 2
7     }
8   ]
9 }
```

Headers 201 Created • 244 ms • 524 B • ...

Key	Value
Content-Type	application/json; charset=utf-8
Date	Fri, 12 Jun 2026 00:22:42 GMT
Server	Kestrel
Location	/api/orders/742aa597-94a7-4962-ad9b-728a3935be3f
Transfer-Encoding	chunked
X-Correlation-Id	QA-TRACE-001

TC-OBS-003 - Auditar operaciones de escritura

- **Identificador:** TC-OBS-003
- **Requisito/Escenario asociado:** Auditoría de **POST**, **PUT** y **DELETE**
- **Precondiciones:** Servicio con logging habilitado y acceso al archivo/log de auditoría.
- **Datos de entrada:** Ejecutar una operación **POST** o **PUT** válida.
- **Pasos:**
 1. Ejecutar una operación de escritura, por ejemplo crear un producto.
 2. Revisar los logs del servicio.
- **Resultado esperado:** Existe una entrada de auditoría con método, path, status code, request y response body.
- **Resultado obtenido:**



TC-OBS-004 - Verificar estado saludable del servicio

- **Identificador:** TC-OBS-004
- **Requisito/Escenario asociado:** Health checks en `/health`
- **Precondiciones:** Servicio objetivo en ejecuci\u00f3n con su base accesible.
- **Datos de entrada:** Request `GET /health`.
- **Pasos:**
 1. Enviar `GET` a `http://localhost:5001/health`.
 2. Revisar el JSON retornado.
- **Resultado esperado:** Respuesta `200` con informaci\u00f3n de runtime, uptime y estado de conectividad de persistencia.
- **Resultado obtenido:**

POST http://localhost:5003/GET http://localhost:5001/h+

No environment

http://localhost:5001/health

SaveShare

GET

http://localhost:5001/health

Send

DocsParamsAuthorizationHeaders 6BodyScriptsTestsSettings

Cookies

Query Params

Key	Value	Description	Bulk Edit	
Key	Value	Description		

BodyCookiesHeaders 8Test Results

200 OK6 ms748 B

{ } JSON

PreviewVisualize

```
1 {
2   "status": "Healthy",
3   "totalDuration": "00:00:00.0001936",
4   "entries": {
5     "persistence-check": {
6       "data": {},
7       "description": "Base de datos SQLite operativa y respondiendo correctamente.",
8       "duration": "00:00:00.0000598",
9       "status": "Healthy",
10      "tags": [
11        "database"
12      ]
13    },
14    "api-status": {
15      "data": {
16        "runtime": ".NET 8.0.27",
17        "uptime": "01:42:30",
18        "startedAt": "2026-06-11T23:21:05.7301978Z"
19      },
20      "description": "API operativa - .NET 8.0.27",
21      "duration": "00:00:00.0000481",
22      "status": "Healthy",
23      "tags": [
24        "api"
25      ]
26    }
27  }
28 }
```